Assignment for Data Analyst position

Data Analysis & Forecasting documentation

By Aiden Pham Dang Khoa

November 2022

Contents

1. Introduction

1.1 Datasets

The raw data includes:

- 11 csv + 3 parq files: order level data from 11 Feb 2019 to 2 Feb 2022.
  Each file contains order data of a tenant.
  Total: 14 columns x 1 428 652 records

	order_date
min	2019-02-11
max	2022-02-02

```
Int64Index: 1428652 entries, 0 to 433
Data columns (total 14 columns):
 #   Column                   Non-Null Count    Dtype
---  ------                   --------------    -----
 0   order_date               1428652 non-null  object
 1   company_id               1428652 non-null  int64
 2   tenant_id                1428652 non-null  int64
 3   order_id                 1428652 non-null  int64
 4   order_line_id            1428652 non-null  int64
 5   product_category_name    1375077 non-null  object
 6   product_id               1428652 non-null  int64
 7   channel_id               1428652 non-null  int64
 8   order_status_id          1428652 non-null  int64
 9   quantity                 1428652 non-null  int64
 10  OriginalUnitPriceInclVat 1428652 non-null  float64
 11  OriginalUnitVat          1428652 non-null  float64
 12  CurrencyCode             1428652 non-null  object
 13  EuroExchangeRate         1328789 non-null  float64
dtypes: float64(3), int64(8), object(3)
memory usage: 163.5+ MB
```

- 4 json files: dimension data (company, tenant, channel, order status) that can be matched with the id columns in order data.

1.2 Research questions

This assignment aims to answer 2 main questions:

1. **How can we measure the past performance for our clients?**
   a. How does performance fluctuate over time?
   b. What are the best performing channels?
   c. What are the best performing product categories?
   d. What does the distribution of order status look like?
   e. How does price change over time and does it affect performance?

2. **How can we forecast future trends for our clients?**
   a. How can ARIMA model be used to forecast future order quantity?
   b. How can we find the best parameters for the ARIMA model?
   c. How can we implement seasonality in the ARIMA model?
   d. What are the limitations and how can we improve our forecasting model in the future?

1.3 Business significance

Measuring past performance is important for our clients:

- Notice patterns of orders/quantities/revenue over time → understand trend and seasonality
- Benchmark channels → understand which channel brings the most orders & revenues

- Benchmark product categories → understand which categories brings the most orders & revenues
- Monitor order status → take necessary actions (quality check of products, ordering & shipping processes, customer service) if there are too many orders cancelled or returned
- Monitor price change and see its impact on performance → future actions for pricing

Forecasting future trends can help our clients to:
- Plan their product inventory
- Plan manpower to handle the orders
- Estimate potential sales of different product categories in order to allocate their investment

1.4 Methods

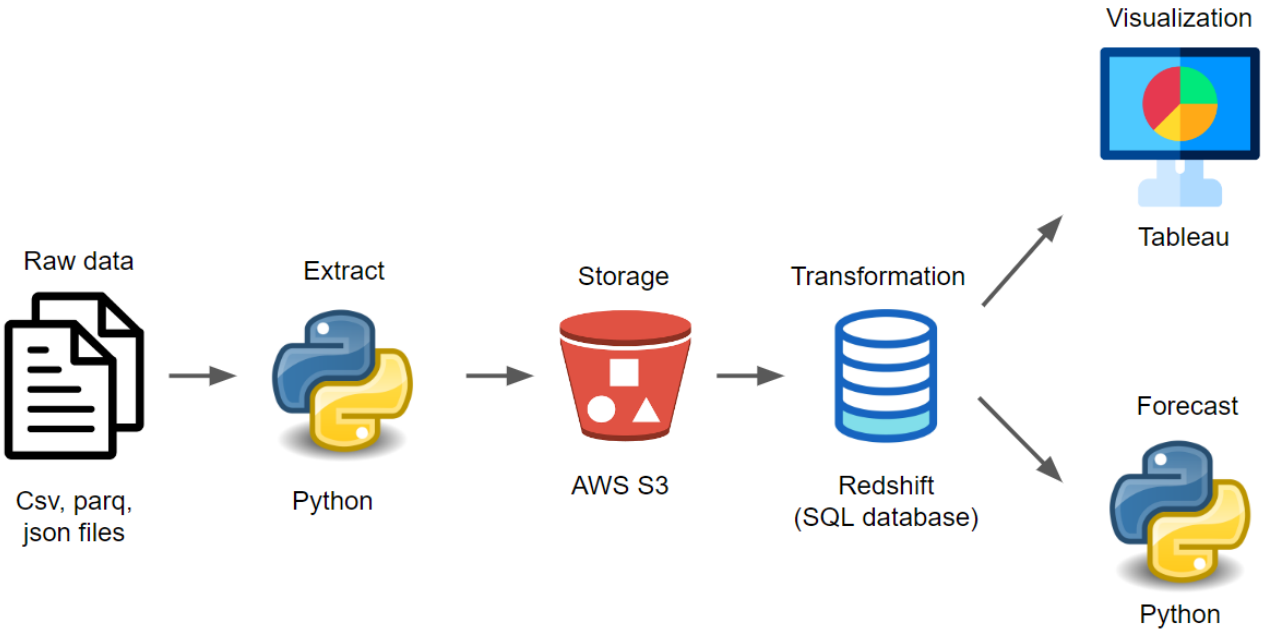
To investigate data quality, we will look at different dimensions of data quality framework: Completeness, Uniqueness, Consistency & Accuracy.

Regarding the measurement of historical performance, data visualisation will be used to examine the performance on different dimensions such as time, channel, categories. Besides, order status and price will also be analysed.

Regarding the forecasting, ARIMA, which is a popular time-series forecasting model, will be used to predict future order quantities.

2. Data Cleaning

2.1 Data pipeline:



Step 1:
Extract the data from csv, parq & json files by using Python.
Also do some initial quality check before saving the data: check number of columns, column names, whether each tenant file has only 1 tenant_id and 1 company_id, etc.
Please check the attached Jupyter notebook file for Python code.

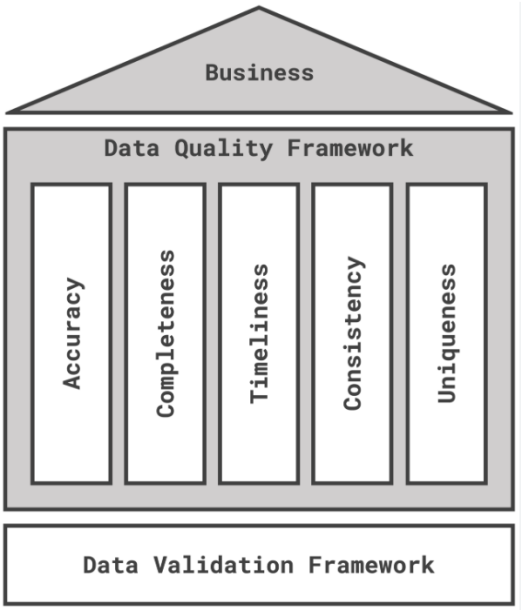
Step 2:
5 parquet files are saved from step 1 and uploaded to S3 (storage service of AWS):

- 1 file containing all orders data (combining 11 csv files and 3 parq files)
- 4 files containing dimension data (from 4 json files)

Step 3:
Load data from S3 to 5 tables in SQL database
Conduct further data quality check and data cleaning in SQL.
Prepare training and validation sets for forecasting models.

Step 4:
Visualise data in Tableau.
Forecast future order quantities by Python.

2.2 Data quality issues:



Framework for data quality dimension was taken from the article of Chau (2021).

In the scope of this assignment, I will focus on Completeness, Uniqueness, Consistency & Accuracy.

a) Completeness:

There are totally 1 428 652 rows in the orders table.
2 columns have some Null values:

- *product_category_name:* 53575 Null rows (3.75%)
- *EuroExchangeRate:* 99863 Null rows (6.99%)

→ to be corrected

order_date	0	order_date	0.000000
company_id	0	company_id	0.000000
tenant_id	0	tenant_id	0.000000
order_id	0	order_id	0.000000
order_line_id	0	order_line_id	0.000000
product_category_name	53575	product_category_name	3.750038
product_id	0	product_id	0.000000
channel_id	0	channel_id	0.000000
order_status_id	0	order_status_id	0.000000
quantity	0	quantity	0.000000
OriginalUnitPriceInclVat	0	OriginalUnitPriceInclVat	0.000000
OriginalUnitVat	0	OriginalUnitVat	0.000000
CurrencyCode	0	CurrencyCode	0.000000
EuroExchangeRate	99863	EuroExchangeRate	6.990016

b) Uniqueness

- Check whether each csv/parq file has only 1 tenant_id and 1 company_id:

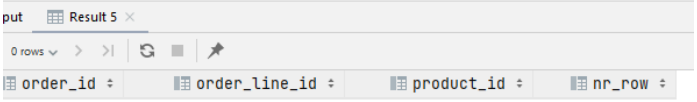
```
nr_company = len(df_temp.company_id.unique())
nr_tenant = len(df_temp.tenant_id.unique())

#check to ensure column names are the same, company and tenant are unique in each file:
if (column_name == column_name_check) & (nr_company == 1) & (nr_tenant == 1):
    df_final = pd.concat([df_final, df_temp])
    nr_rows = len(df_temp.axes[0])
    count_row += nr_rows
    print(file + ' has ' + str(nr_rows) + ' rows and has been added.')
else:
    print(file + ' has error and cannot be added to dataframe.')
```

Result: each file has only 1 tenant_id and 1 company_id.

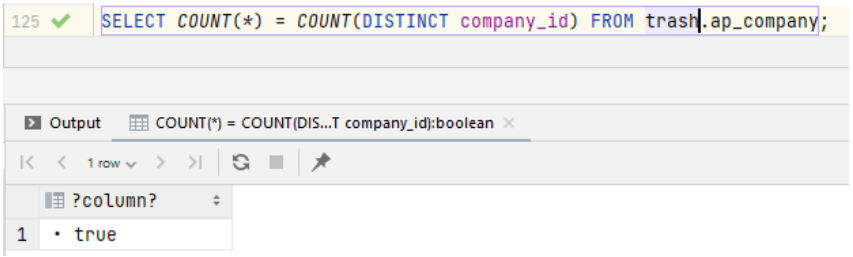
- Check whether the combination of (order_id, product_line_id, product_id) is unique in **orders table**:

```
SELECT
  order_id,
  order_line_id,
  product_id,
  COUNT(*) AS nr_row
FROM
  trash.ap_all_orders
GROUP BY
  1,
  2,
  3
HAVING nr_row > 1;
```



Result: no records returned for above query → confirmed unique

- Similarly, check whether the id in each **dimension table** is unique (for example, each company_id should appear only once in company table):



Result: confirmed that all 4 dimension tables have unique id column

c) Consistency

- Check whether company_id in **orders table** is consistent with the **tenant dimension table**:

```
172 ✓ select count(*) from trash.ap_all_orders o
173   left join trash.ap_tenant t using (tenant_id)
174   where o.company_id != t.company_id
175   ;
```

Output | count(*):bigint

1 row

	count
1	0

Result: consistent

- Check if all company_id, tenant_id, channel_id, order_status_id in **orders table** have a matching values in **dimension tables**

```
172 ✓ SELECT
173     COUNT(*)
174   FROM
175     trash.ap_all_orders o
176     LEFT JOIN trash.ap_tenant t USING (tenant_id)
177     LEFT JOIN trash.ap_company c ON o.company_id = c.company_id
178     LEFT JOIN trash.ap_channel ch ON o.channel_id = ch.pins_plugininfoid
179     LEFT JOIN trash.ap_order_status os ON o.order_status_id = os.id
180   WHERE
181     t.tenant_code IS NULL OR c.company_name IS NULL OR ch.plugininfo_name IS NULL
182                           OR os.status IS NULL
183   ;
```

Output | COUNT(*):bigint

1 row

	count
1	0

Result: all have matching id

d) Accuracy

- **Check min and max value of each column:**

	order_date	company_id	tenant_id	order_id	order_line_id	product_id	channel_id
min	2019-02-11	223	1736	2880	36385595	14890762	1
max	2022-02-02	973	9305	9999992659	99999843855904	9999914902180	1532

order_status_id	quantity	OriginalUnitPriceInclVat	OriginalUnitVat	CurrencyCode	EuroExchangeRate
0	-147	0.000	0.000	EUR	0.8179
10	90	3659.916	731.984	USD	1.2025

```
501 ✔  select * from trash.ap_all_orders where quantity < 0
```

Output | product_id | trash.ap_all_orders

55 rows | Tx: Auto

_name	product_id	channel_id	order_status_id

Result: Other columns look ok, but the quantity column has 55 negative values. A further look at these 55 rows reveals that they have the same company (Jelly BV), tenant (Jelly b2b), channel (Kaufland) & orderstatus (cancelled).
→ to change to 0 quantity (the risk is low since status is cancelled and there are only 55 records)

- **Check if EuroExchangeRate column makes sense:**

Result: Except for PLN and SEK, the exchange rate of other currencies look reasonable.

```
517 ✔  SELECT
518      currencycode,
519      COUNT(*),
520      MIN(euroexchangerate),
521      MAX(euroexchangerate),
522      MEDIAN(euroexchangerate)
523  FROM
524      trash.ap_all_orders
525  GROUP BY
526      1;
```

Output | Result 2

5 rows

	currencycode	count	min	max	median
1	SEK	482	1	1	1
2	PLN	2423	1	1	1
3	USD	2228	0.8179	1	0.855
4	GBP	29100	1	1.2025	1
5	EUR	1394419	1	1	1

Check if any rows have currencies other than EUR but exchange rate = 1:

```sql
SELECT COUNT(*)
FROM
  trash.ap_all_orders
WHERE currencycode != 'EUR' AND euroexchangerate = 1;
```

Output | product_id | COUNT(*):bigint

1 row

	count
1	25911

```sql
SELECT DISTINCT
  currencycode,
  tenant_code,
  COUNT(*)
FROM
  trash.ap_all_orders
  LEFT JOIN trash.ap_tenant USING (tenant_id)
WHERE currencycode != 'EUR' AND euroexchangerate = 1
GROUP BY 1, 2;
```

Output | product_id | Result 41

10 rows

	currencycode	tenant_code	count
1	GBP	Jelly b2b	20095
2	GBP	Bitterballen BV	2803
3	GBP	Sofa team BV	103
4	GBP	Acme Corp EMEA	2
5	PLN	Jelly b2b	327
6	PLN	Jelly b2c	2037
7	PLN	Horizon Zone CA	1
8	PLN	Bitterballen BV	58
9	SEK	Jelly b2b	482
10	USD	Acme Corp APAC	3

Result: about 26k rows (1.81%) have this suspicious condition. USD and GBP may have exchange rates close to 1. However, PLN (Poland) and SEK (Sweden) currencies cannot have an exchange rate of 1 (from 2019 to 2022, 1 PLN is about 0.22 EUR and 1 SEK is about 0.095 EUR). 2905 rows (0.2%) have PLN and SEK currency with an exchange rate of 1.

This is harder to correct because we are not sure whether the currency column is wrong or the exchange rate column is wrong.

→ One possible action is to check with the below clients whether they listed their products in PLN and SEK currency:

```sql
SELECT DISTINCT
  company_name,
  tenant_code,
  currencycode,
  COUNT(*)
FROM
  trash.ap_all_orders_2 o
  LEFT JOIN trash.ap_company c USING (company_id)
  LEFT JOIN trash.ap_tenant t USING (tenant_id)
WHERE currencycode IN ('PLN', 'SEK')
GROUP BY 1,2,3;
```

Output | Check which company ...PLN and SEK currency:

5 rows

	company_name	tenant_code	currencycode	count
1	Bitterballen Fashion	Bitterballen BV	PLN	58
2	Horizon Zone	Horizon Zone CA	PLN	1
3	Jelly BV	Jelly b2b	SEK	482
4	Jelly BV	Jelly b2b	PLN	327
5	Jelly BV	Jelly b2c	PLN	2037

2.3 Data correction

The following actions need to be done for our dataset:

- Replace negative **quantity** values with 0
- Replace Null **product_category_name**
- Replace Null **EuroExchangeRate**

- ***quantity:*** change 55 negative values to 0

- ***product_category_name:***
  Replace Null value by checking product_category_name of other rows with the same **tenant id + channel id + product id**

  The below query returns no rows, meaning that we can use the combination of **tenant id + channel id + product id** to replace Null value of product_category_name (In contrast, using **product id** alone is not good because a quick check reveal that each **product id** can have more than 1 **product_category name**).

```sql
SELECT
  product_id,
  channel_id,
  tenant_id,
  COUNT(DISTINCT product_category_name) AS nr
FROM
  trash.ap_all_orders o
WHERE
    product_category_name IS NOT NULL
AND product_id IN (
  SELECT DISTINCT
    product_id
  FROM
    trash.ap_all_orders
  WHERE product_category_name IS NULL
                  )
AND channel_id IN (
  SELECT DISTINCT
    channel_id
  FROM
    trash.ap_all_orders
  WHERE product_category_name IS NULL
  )
AND tenant_id IN (
  SELECT DISTINCT
    tenant_id
  FROM
    trash.ap_all_orders
  WHERE product_category_name IS NULL
  )
GROUP BY
  1, 2, 3
HAVING nr > 1;
```

- ***EuroExchangeRate****:*

- Most rows with Null *EuroExchangeRate* actually has EUR currency, which can be easily corrected (replace Null with 1.0)
- Other rows with Null *EuroExchangeRate* has GBP currency, so we can find the exchange rate by checking other rows with GBP currency code (on the same order_date)

```python
# check rows with Null EuroExchangeRate
df_temp = df_final[df_final.EuroExchangeRate.isnull()]
df_temp
```

```python
# check what currency does not have exchange rate:
df_temp['CurrencyCode'].value_counts()

# most are EUR, which should have Exchange rate 1.0
# some are GBP. Missing ExchangeRate can be found by taking exchange rate from rows with GBP CurrencyCode (on same orde
```

```
EUR    94904
GBP     4959
Name: CurrencyCode, dtype: int64
```

```sql
SELECT
  currencycode,
  order_date,
  COUNT(DISTINCT euroexchangerate) AS nr,
  MIN(euroexchangerate) AS min_exchange,
  MAX(euroexchangerate) AS max_exchange
FROM
  trash.ap_all_orders
GROUP BY
  1,
  2
HAVING nr > 1;
```

Output | product_id | Result 43

452 rows

	currencycode	order_date	nr ▼ 1	min_exchange	max_exchange
1	GBP	2021-08-09	4	1	1.1797
2	GBP	2021-08-05	3	1	1.1739
3	GBP	2021-03-02	3	1	1.1552
4	GBP	2021-09-05	3	1	1.167
5	GBP	2020-12-12	3	1	1.0976
6	GBP	2021-08-26	3	1	1.1689

The above picture shows that some currencies may have more than 1 exchange rate per day. Min and Max values are within a reasonable range (0.8 to 1.2), so there are no abnormal outliers. We can use average value (per currency per day) to replace Null exchange rate.In case there are many outliers, we should use median instead of average.

- **Checking after correction:**
  After replacing Null *product_category_name* and *EuroExchangeRate,* make sure we do not create any duplicates (the number of rows should remain unchanged).

  Check again to see if there are still any Null values:

  - There are still 53575 records (3.75%) with Null *product_category_name.* Majority of the Null values come from Bitterballen BV

```sql
SELECT * FROM trash.ap_all_orders WHERE product_category_name IS NULL;
```

Output | product_id | trash.ap_all_orders

1-500 of 53,575

```sql
SELECT
  company_name,
  tenant_code,
  COUNT(*)
FROM
  trash.ap_all_orders
  LEFT JOIN trash.ap_company USING (company_id)
  LEFT JOIN trash.ap_tenant USING (tenant_id)
WHERE product_category_name IS NULL
GROUP BY
  1,
  2;
```

Output | product_id | Result 62

6 rows

	company_name	tenant_code	count
1	BuYaYa BV	BuYaYa BV	1209
2	Bitterballen Fashion	Bitterballen BV	51720
3	Acme Corp	Acme Corp US	2
4	Horizon Zone	Horizon Zone CA	6
5	Horizon Zone	Horizon Zone EMEA	28
6	Green is good	Green is good BV	610

- There are 4918 records (0.34%) with GBP currency having Null *EuroExchangeRate* (because we cannot find rows with same currency on same day)

```sql
SELECT * FROM trash.ap_all_orders WHERE euroexchangerate IS NULL;
```

Output | product_id | trash.ap_all_orders

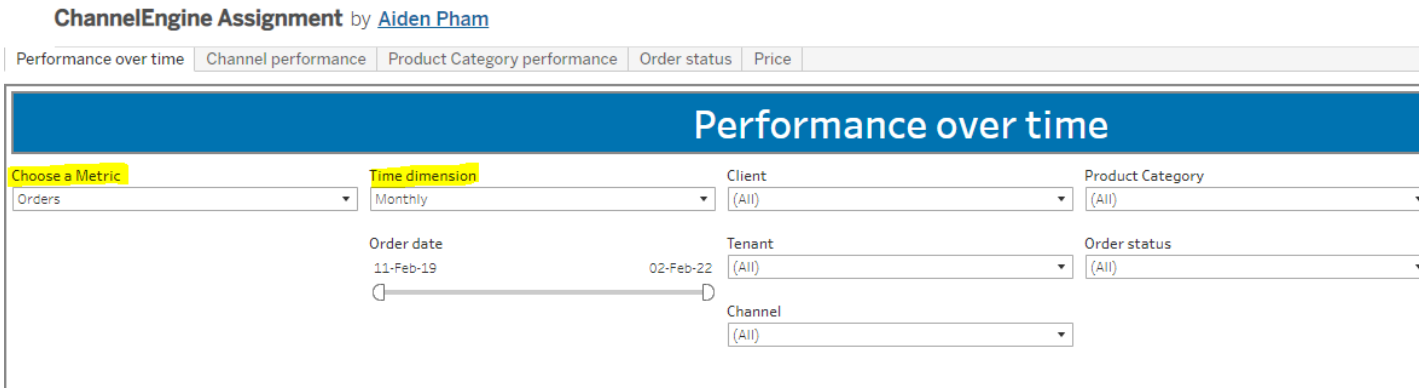
1-500 of 4,918

→ using avg exchange rate per month of that currency → number of Null decreases to 1932 (0.14%). The remaining records with Null exchange rate are from 16 Feb 2020 to 31 Mar 2020

→ to replace with a fixed value of 1.1 (exchange rate GBP/EUR was about 1.1 in February - March 2020) → number of Null exchange rate goes to 0.

3. Data analysis:

Data analysis results are summarised in the presentation slides.
Link to visualisation:
https://public.tableau.com/app/profile/aiden.pham/viz/ChannelEngineAssignment/Performanceovertime?publish=yes

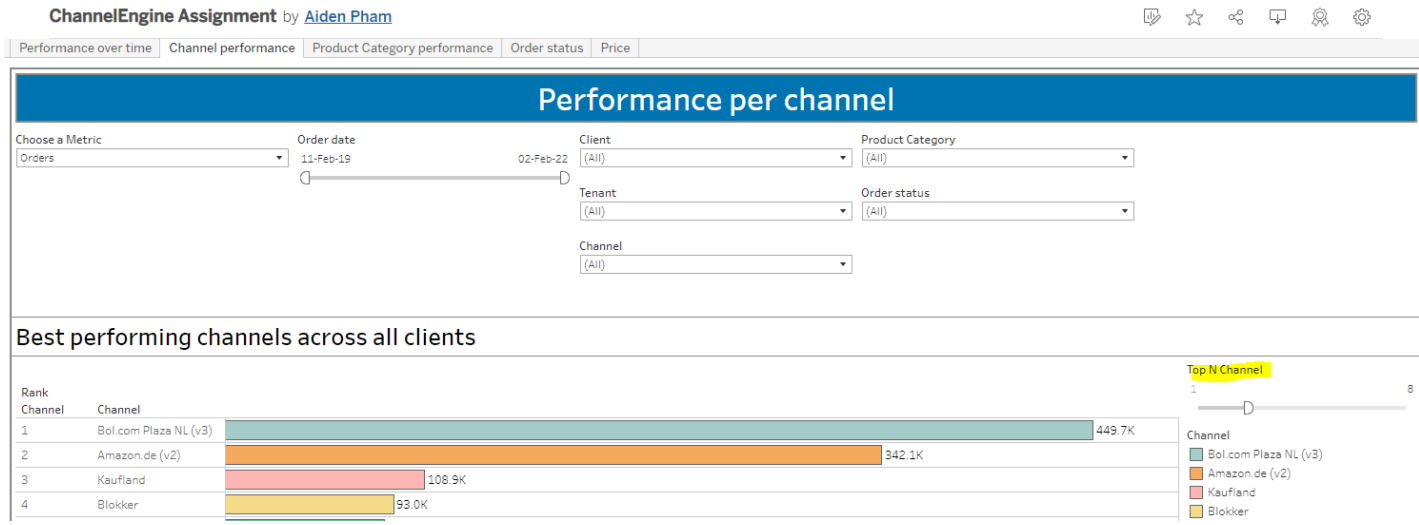


The ‘Choose a Metric” filter allows user to switch between:

- Number of orders
- Total order quantity
- Total revenue (in EUR)

The ‘Time dimension” filter allows user to switch between different time aggregation:

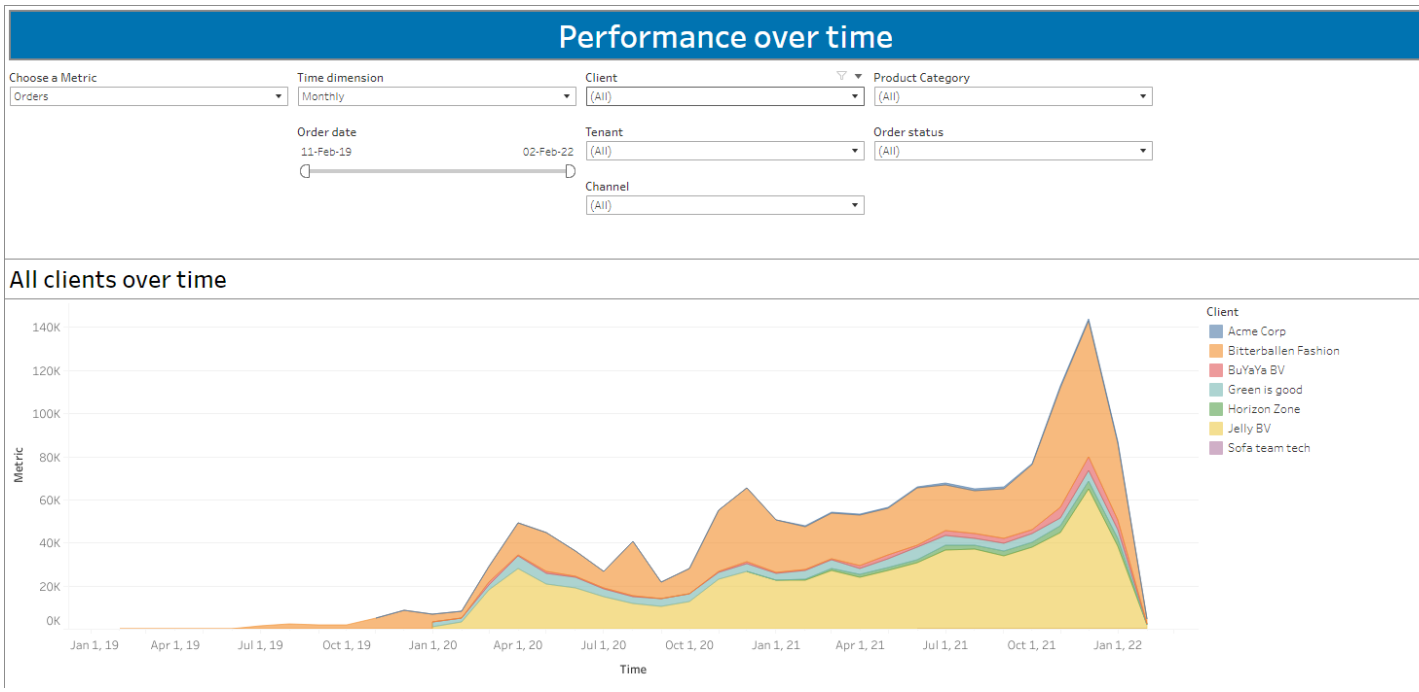
- Daily
- Weekly
- Monthly
- Quarterly
- Yearly



In “Channel performance” and “Product Category performance” tabs, there are “Top N” filters, which allow users to select the number of top results they want to see (eg. Set “Top N Channel” to 5 will show 5 best performing channels)

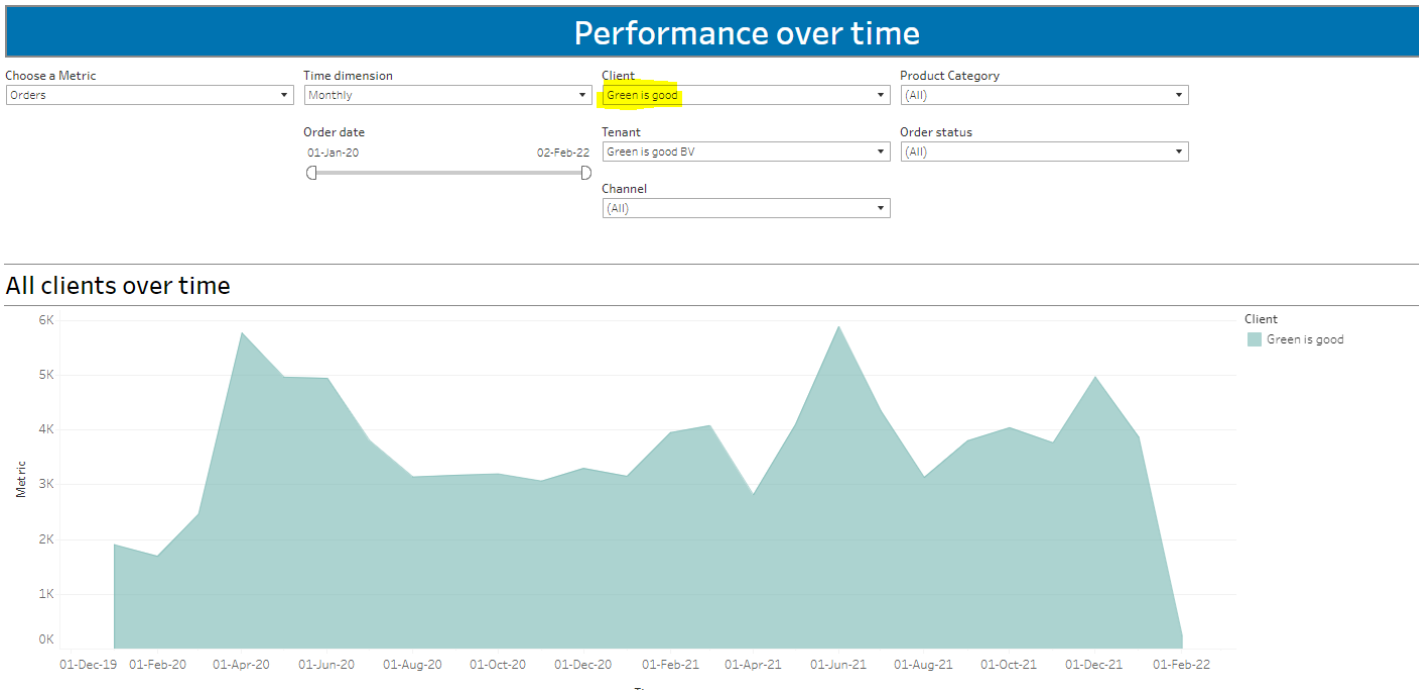
3.1 Performance over time:

Total orders/quantity/revenue (sum of all clients) increase significantly from 2019 to 2022.



Bitterballen fashion and Jelly BV are the two biggest clients in terms of orders/quantity/revenue.

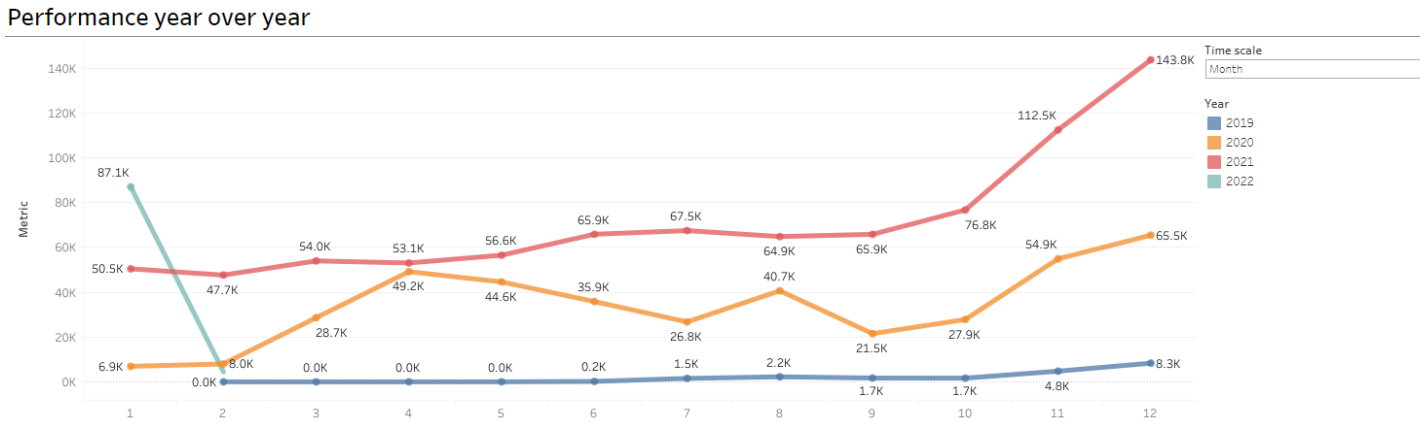
Overall, orders/quantity/revenue increase over time for all clients, except for “Green is good”. The performance of “Green is good” is relatively stable and does not increase as quickly as other clients.



December has the best performance every year. This can be attributed to 2 main reasons:

- Seasonality due to Christmas/New Year holiday seasons

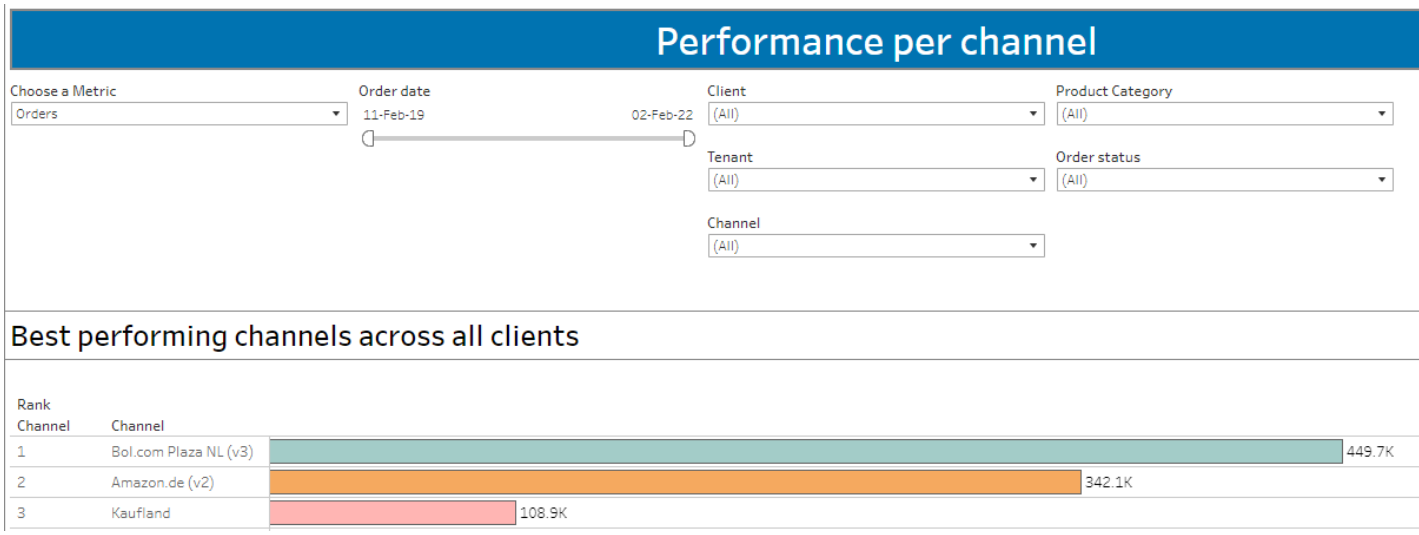
- Overall increasing trend from January to December



There was a spike in April 2020 probably related to the 1st corona lockdown.
Data for February 2022 seems low because we only have order data until 2 Feb 2022, so we count for only 2 days of February 2022.

3.2 Channel:

The best performing channel overall is **Bol.com Plaza NL**. **Amazon.de** is in 2nd place, and **Kaufland** is in 3rd place. The top 3 does not change no matter what metric we choose (orders or quantity or revenue).



For each client and each tenant, the top 3 best-performing channel is different:

Best performing channels per client

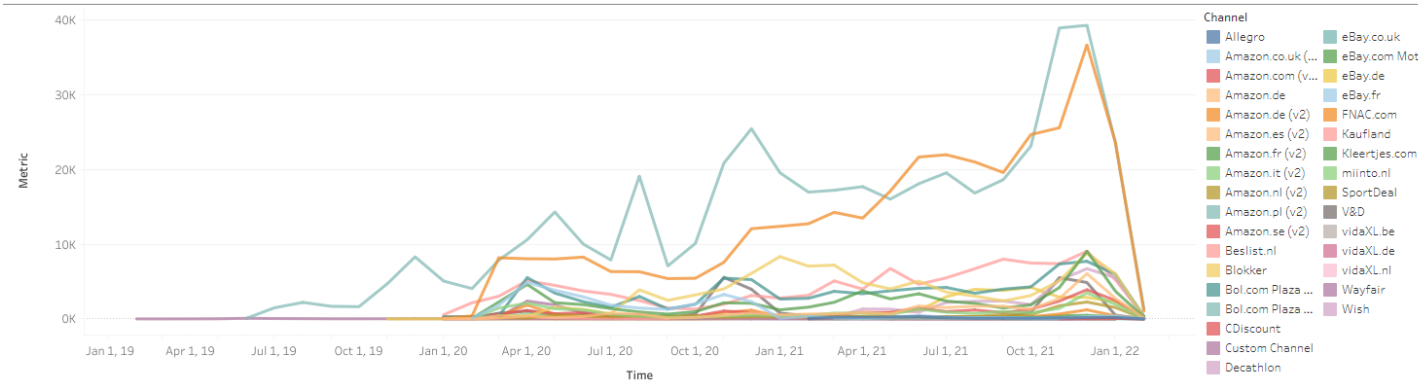
Rank of Client	Client	Rank Channel	Channel	
1	Jelly BV	1	Amazon.de (v2)	266.9K
		2	Kaufland	97.7K
		3	Blokker	59.3K
2	Bitterballen Fashion	1	Bol.com Plaza NL (v3)	380.4K
		2	Amazon.de (v2)	67.3K
		3	Bol.com Plaza BE (v3)	50.2K
3	Green is good	1	Bol.com Plaza NL (v3)	69.1K
		2	Bol.com Plaza BE (v3)	15.6K
		3	Amazon.de (v2)	5.5K
4	BuYaYa BV	1	Decathlon	26.4K
		2	Blokker	4.2K
		3	V&D	3.1K

Best performing channels per tenant

Rank of Client	Client	Rank of Tenant	Tenant	Rank Channel	Channel	
1	Jelly BV	1	Jelly b2b	1	Amazon.de (v2)	266.9K
				2	Kaufland	97.7K
				3	Blokker	59.3K
		2	Jelly b2c	1	eBay.de	22.3K
				2	Decathlon	3.9K
				3	Allegro	1.9K
2	Bitterballen Fashion	1	Bitterballen BV	1	Bol.com Plaza ..	380.4K
				2	Amazon.de (v2)	67.3K
				3	Bol.com Plaza ..	50.2K
3	Green is good	1	Green is good BV	1	Bol.com Plaza ..	69.1K
				2	Bol.com Plaza ..	15.6K
				3	Amazon.de (v2)	5.5K

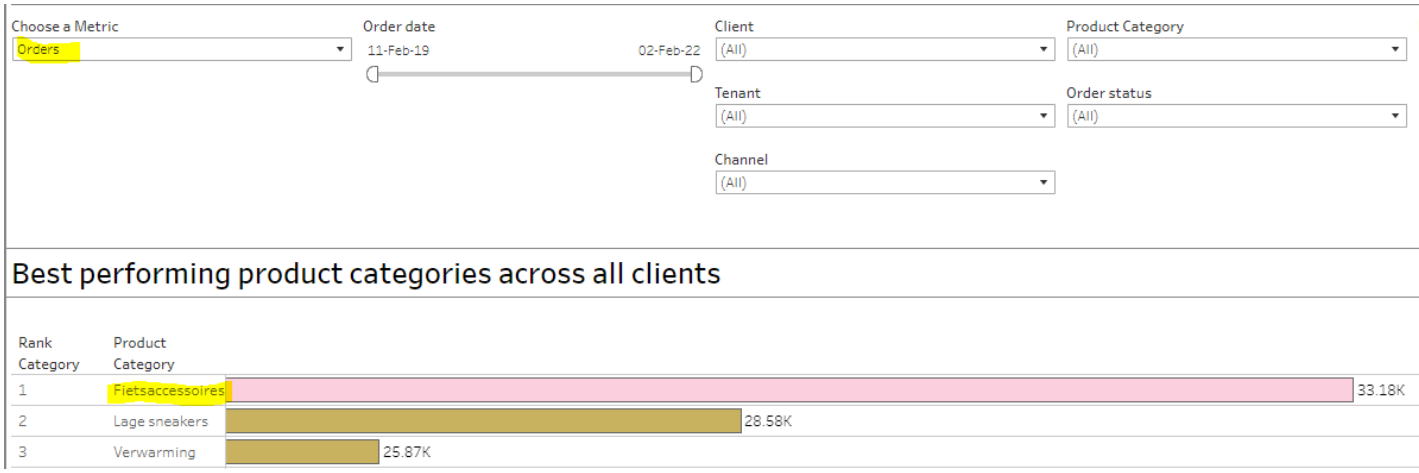
Best-performing channels changed over time. **Bol.com Plaza NL** was at number 1 position for almost every month except for summer 2021 when **Amazon.de** overtook it to take the top spot. The 3rd position kept changing over time. Most of the time, the 3rd spot was the competition between **Kaufland** and **Blokker**.

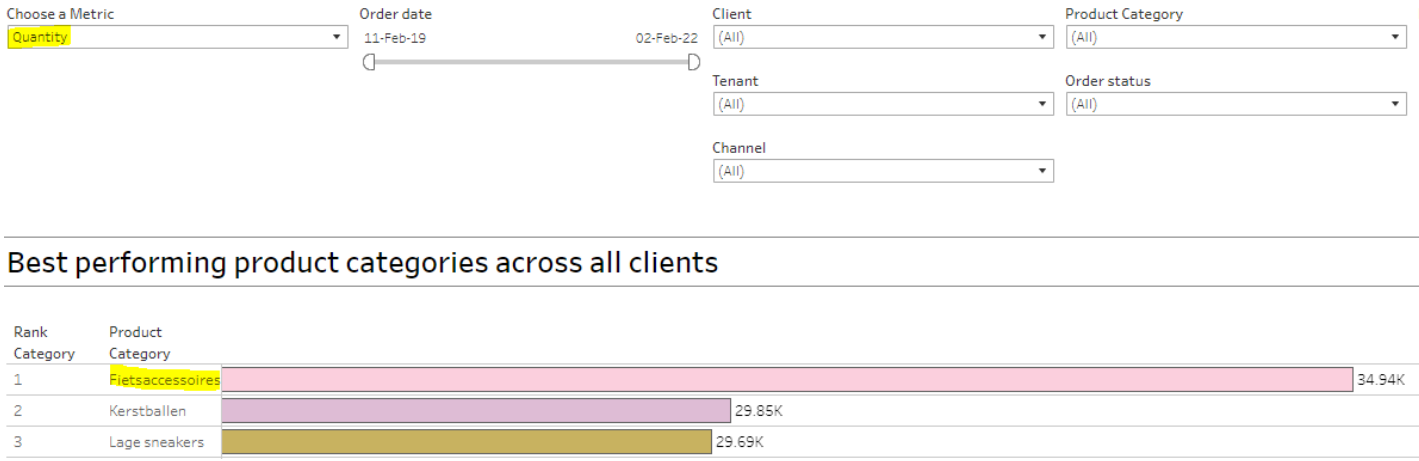
Compare channels over time

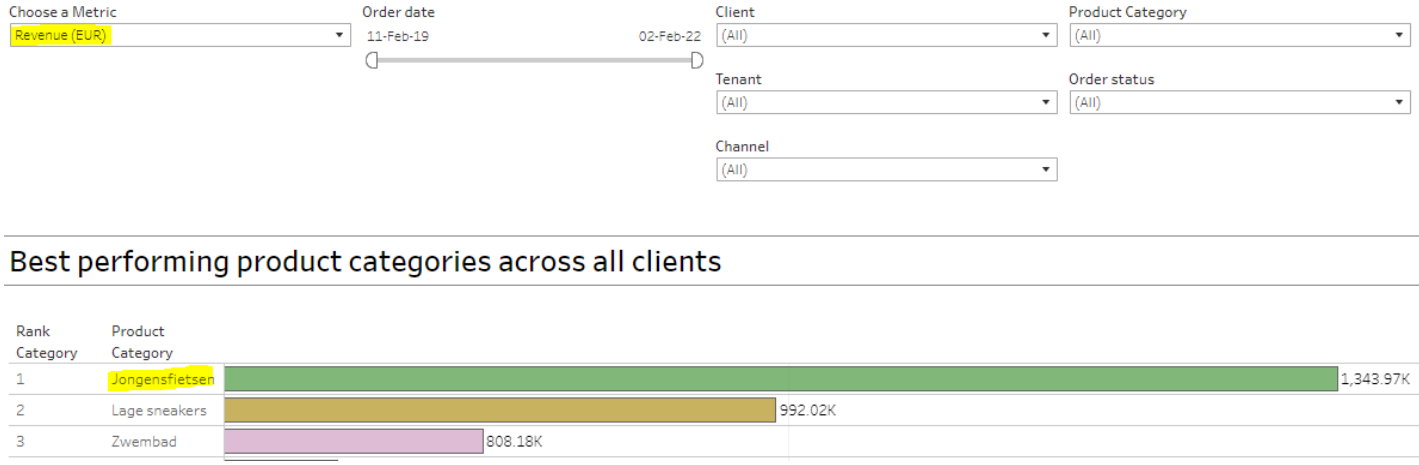


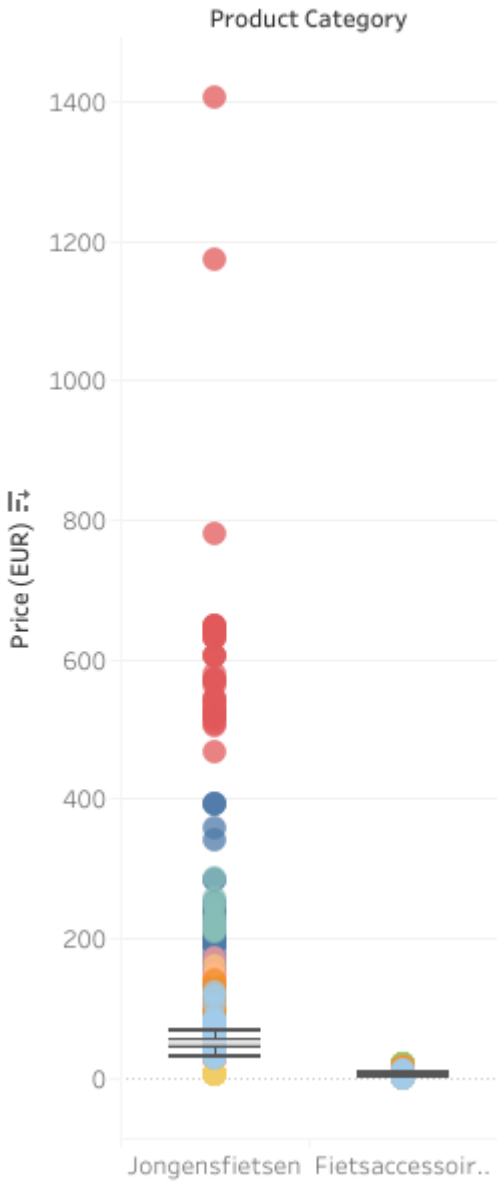
3.3 Product category:

Excluding the Null product category, **Fietsaccesories** takes the top spot in terms of orders & quantity, whereas **Jongenfiets** is number 1 in terms of revenue. This is because **Jongenfiets** has a higher price than **Fietsaccessoires**.





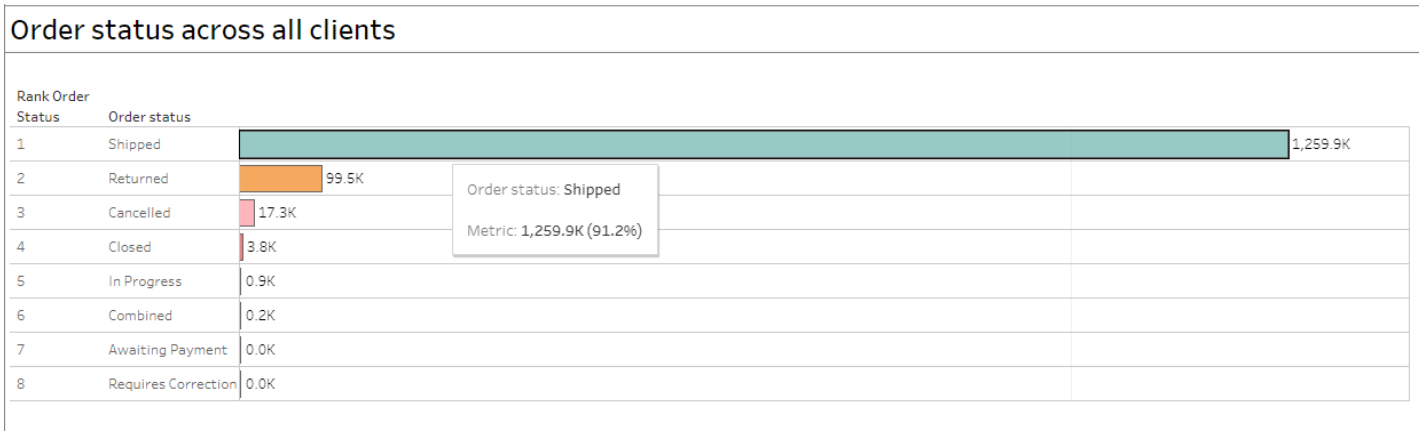




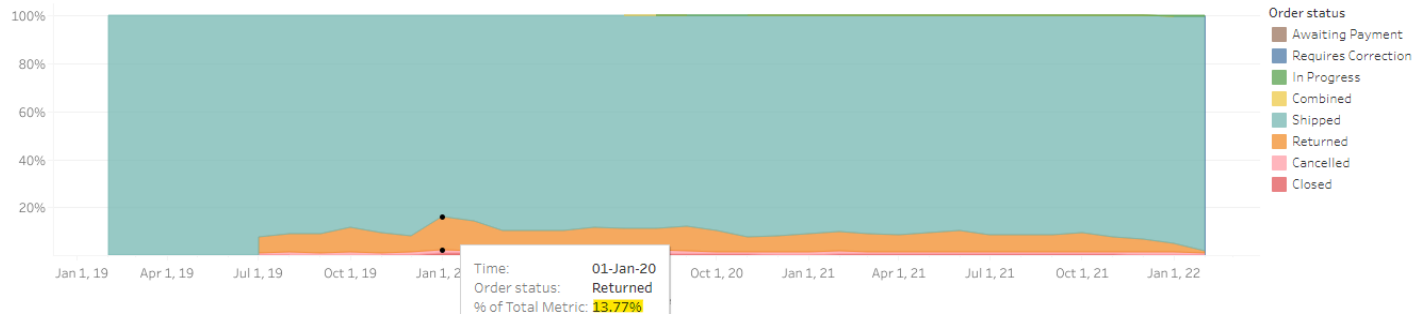
Lagsnakers is in the top 3 for all metrics (orders, quantity, revenue).
Depending on the metric we choose, the best-performing categories for each client and each tenant also change.

3.4 Order status:

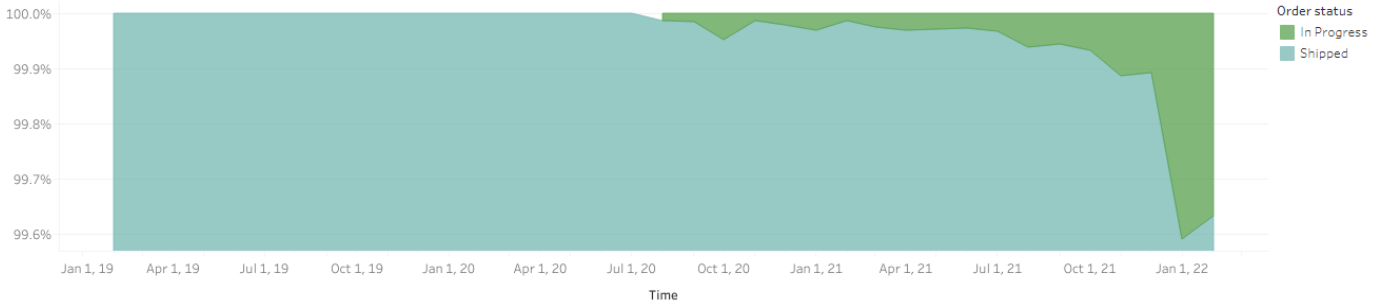
Over 90% of orders have status “Shipped” while about 7% have status “Returned”. 1% of orders were “Cancelled”. The other statuses account for a very small percentage of orders.



% of “Returned” orders decreased over time, from 8%-10% in 2019-early 2020 to about 7% in 2021.



A large portion of orders in the latest months are still “In progress”.



All clients have over 90% orders with status “Shipped”, except for Green is good. This client has only 70% shipped orders. Returned orders make up 26.5% of all orders, which is quite alarming.

Order status per client

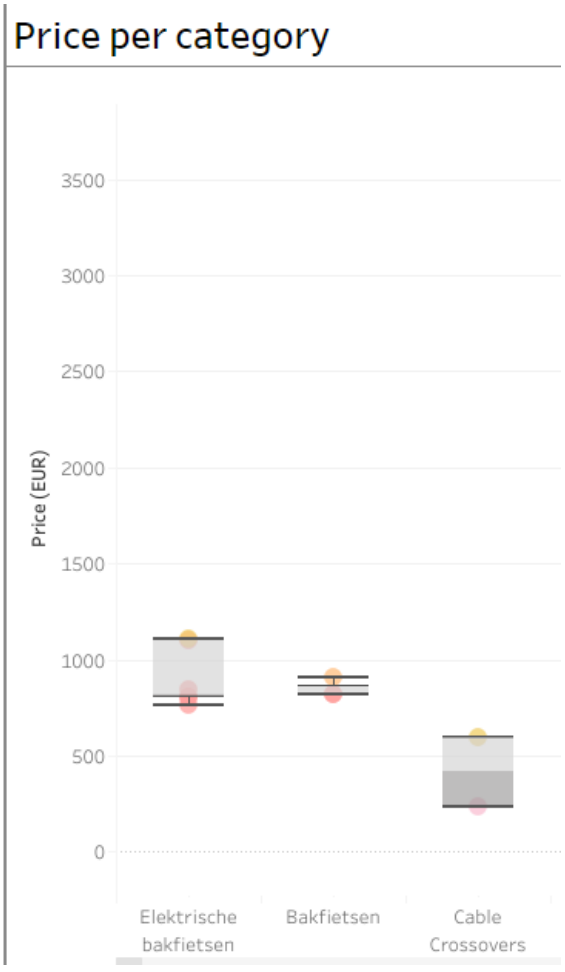
Rank of Client	Client	Rank Order Status	Order status	
3	Green is good	1	Shipped	66.3K
		2	Returned	24.8K
		3	Closed	1.6K
		4	Cancelled	0.9K
		5	In Progress	0.0K
4	BuYaYa BV	1	Shipped	33.4K
		2	Returned	1.1K

Client: Green is good
Channel: Shipped

Metric: 66.3K (70.8%)

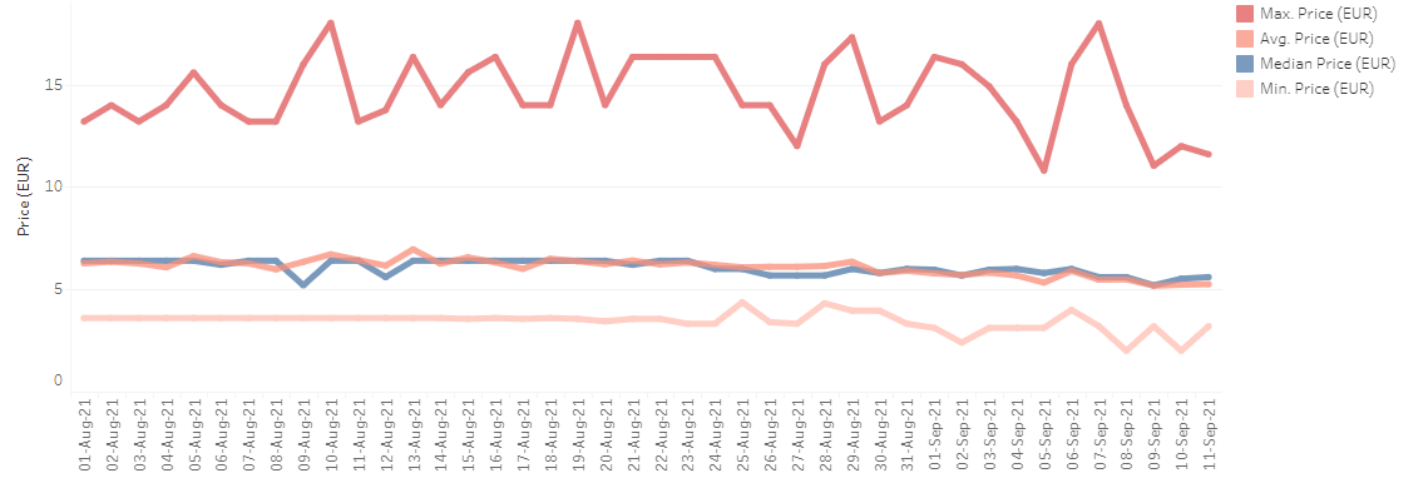
3.5 Price:

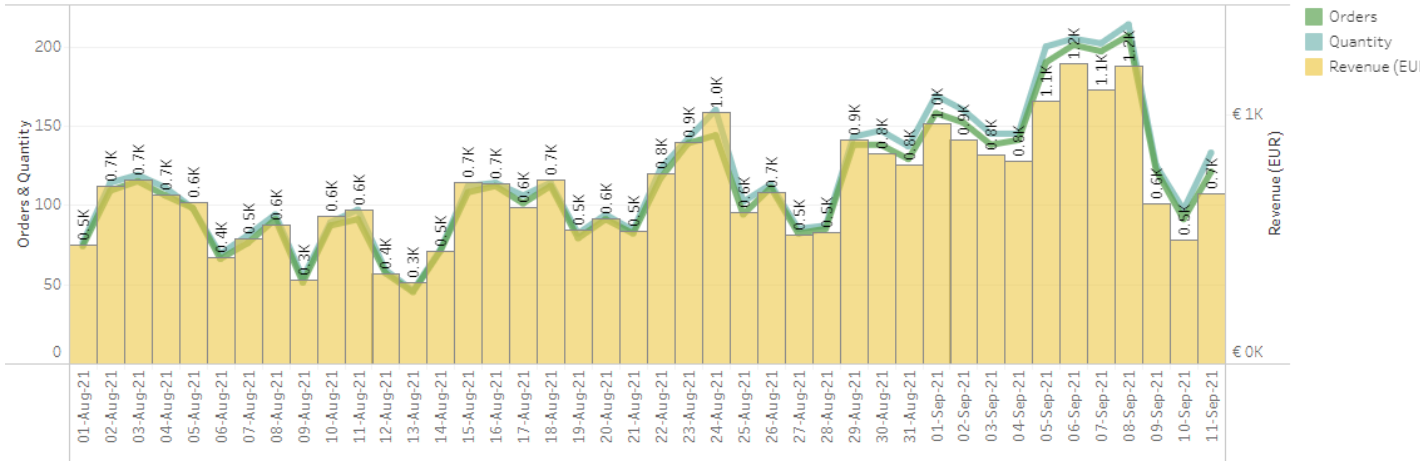
Top 3 categories with highest median price are:



Using the below 2 graphs, we can monitor the change of price over time for each Product category, and study whether price is correlated with orders/quantity/revenue. To check correlation between price and orders/quantity/revenue, it is better to limit the time frame to a few weeks only because long-term trend is influenced by many other factors apart from price (eg. inflation, marketing campaigns, growth of company, general economic conditions, etc.)

For example, for Fietsaccessoires category, price decreased slightly in Aug-Sep 2021. Orders/Quantity/Revenue increased gradually over the same period of time. However, correlation is not equal to causation. Whether this upward trend is due to the drop in price requires further analysis.





4. Forecasting

4.1 Introduction to ARIMA model

AutoRegressive Integrated Moving Average (ARIMA) is a popular model to forecast future value from historical time-series data. ARIMA includes 3 main parts:

- **AR (Autoregression):** measures the dependency of a particular record with a few past observations.

$$Y_t = \alpha + \beta_1 Y_{t-1} + \beta_2 Y_{t-2} + .. + \beta_p Y_{t-p} + \epsilon_1$$

- **I (Integrated):** indicates the number of transformations that is done to make the time-series stationary. Definition of stationary time-series: https://otexts.com/fpp2/stationarity.html
  We need the time-series to be stationary when training the model in order to minimise the obvious correlation with past data and prevent incorrect bias.
  A non-stationary series can be converted to stationary series by several methods. The most popular one is differencing.
- **MA (Moving Average):** size of the moving average window. Instead of using past values like the AR model, a moving average model uses past forecast errors in a regression-like model.

$$Y_t = \alpha + \epsilon_t + \phi_1 \epsilon_{t-1} + \phi_2 \epsilon_{t-2} + .. + \phi_q \epsilon_{t-q}$$

In summary, for ARIMA model:

Predicted Yt = Constant + Linear combination Lags of Y (up to p lags) + Linear Combination of Lagged forecast errors (up to q lags)

A simple (non-seasonal) ARIMA model requires 3 parameters (p, d, q) for training, which correspond to 3 components of ARIMA: AR, I and MA.

- **p:** The number of lag observations included in the model, also called the lag order.
- **d:** The number of times that the raw observations are differenced, also called the degree of difference.
- **q:** The size of the moving average window, also called the order of moving average.

There is Python package that automatically searches for best combination of (p, d, q). We can also manually determine by taking differencing until the series becomes stationary. p and q can be estimated from PACF (Partial Autocorrelation) and ACF (Autocorrelation) graphs of the time-series, respectively.

ARIMA vs SARIMA

Some time-series have strong seasonality (eg. monthly data has a cycle of 12 periods). A seasonal ARIMA (SARIMA) model can be used to train the time-series. In addition to (p, d, q), SARIMA requires some extra parameters:

- **P:** The number of lag observations of the seasonality
- **D:** The number of times that the seasonality is differenced
- **Q:** The size of the moving average window for seasonality
- **M:** the number of periods in a cycle (eg. 12 for monthly data, 4 for quarterly data).

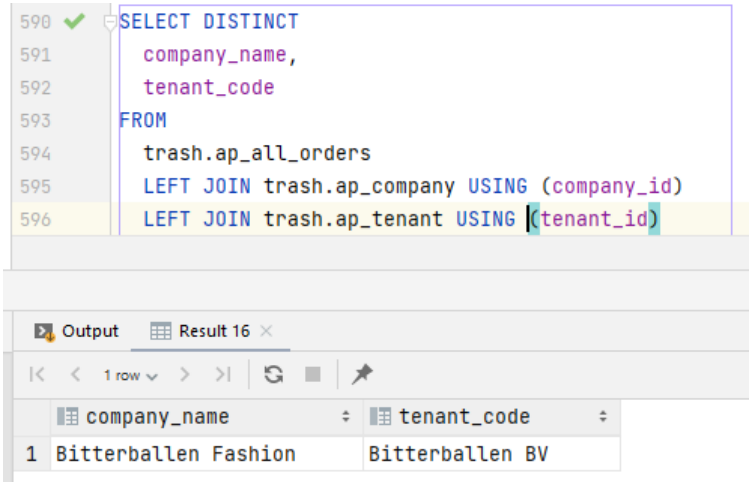
How to determine the values of (P, D, Q) is similar to that of (p, d, q). The only difference is that we look at the seasonality instead of the original time series.

Again, there is a Python package to automate the searching for these parameters. To minimise computation time, it is still recommended to at least check the value of D first, then let the package search for P and Q.

Training and Testing data

We will train and test quantity data because quantity information is important for clients to to prepare their product inventory for potential future sales. “Fietsaccessoires” product category was chosen because:

- It has the highest quantity among the categories (Product with very small quantity has high fluctuation, and is generally more difficult to forecast. Different techniques are required to forecast these small-volume categories (eg. hierarchical forecasting))
- It is sold by 1 tenant only



- It has a low % of cancellation. Excluding cancelled orders will not affect the overall quantity a lot. The other statuses (apart from Cancelled) will be included in the data because the client still needs to send product to customers even if it is returned/ in progress/ combined/ closed.

```sql
SELECT DISTINCT
  status,
  SUM(quantity)
FROM
  trash.ap_all_orders o
  LEFT JOIN trash.ap_order_status os ON o.order_status_id = os.id
WHERE product_category_name LIKE 'Fietsaccessoires'
GROUP BY
  1;
```

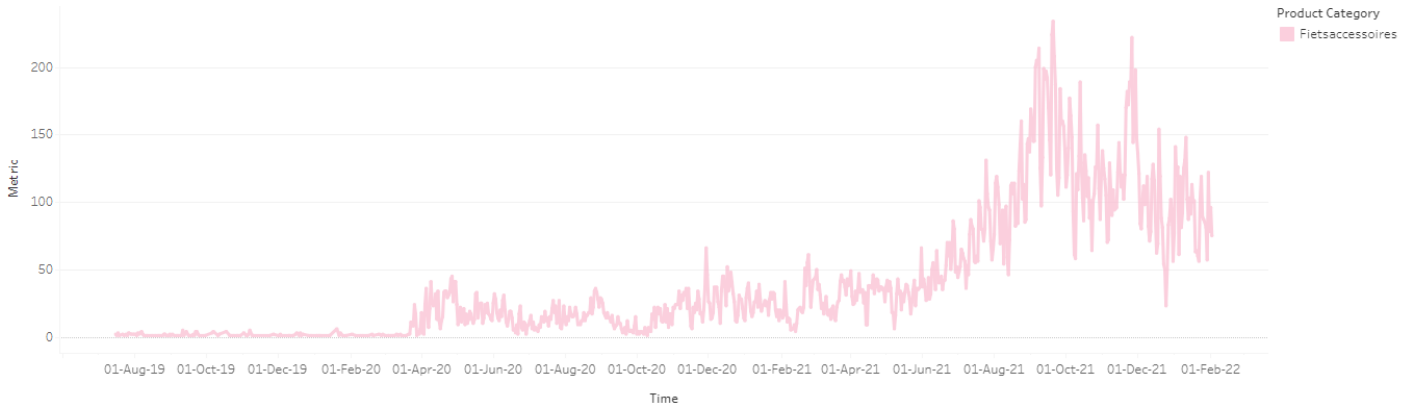
Output | Result 17

6 rows

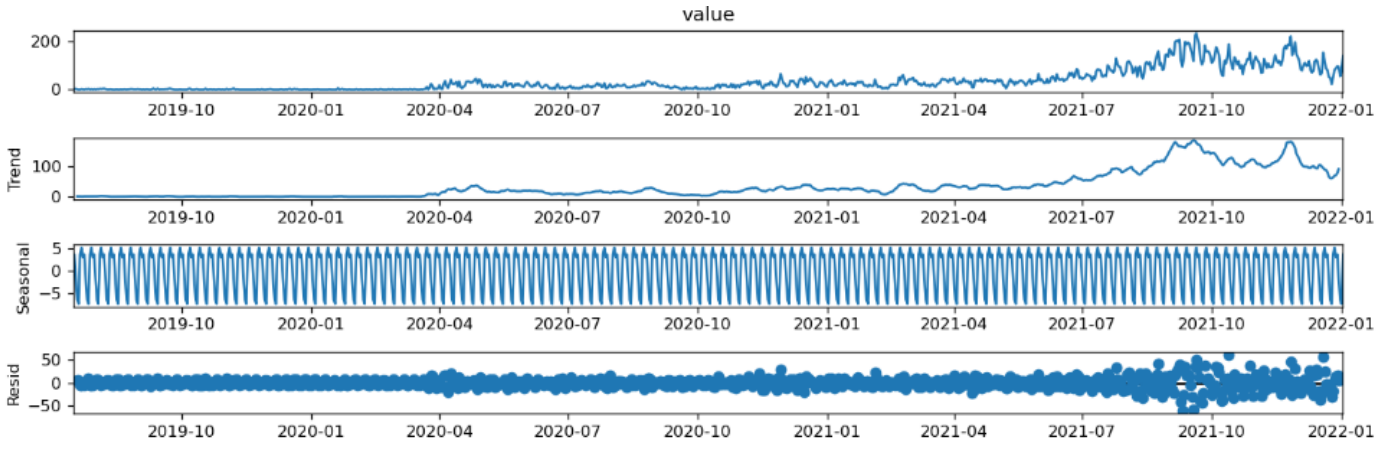
	status	sum 1
1	Shipped	33656
2	Returned	1104
3	Cancelled	139
4	In Progress	20
5	Combined	15
6	Closed	9

We will train the model on daily data of “Fietsaccessoires”. The date ranges from 16 Jul 2019 to 2 Feb 2022:

- Training data is from 16 Jul 2019 to 2 Jan 2022.
- Testing data is from 3 Jan 2022 to 2 Feb 2022.



Decomposition of the time series demonstrates that there is strong seasonality.



Therefore, seasonal ARIMA should be checked in addition to the simple ARIMA.
In this assignment, we will test 4 different ARIMA & SARIMA model on the data of Product Category "Fietsaccessoires":

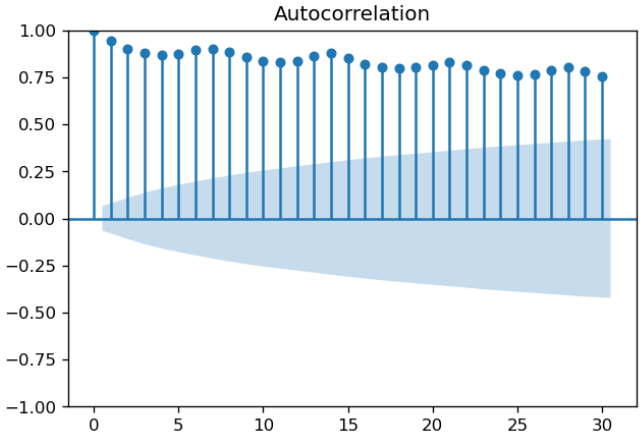
- Non-seasonal ARIMA by manual setting
- Non-seasonal ARIMA by automatic search
- Seasonal ARIMA by manual setting
- Seasonal ARIMA by automatic search

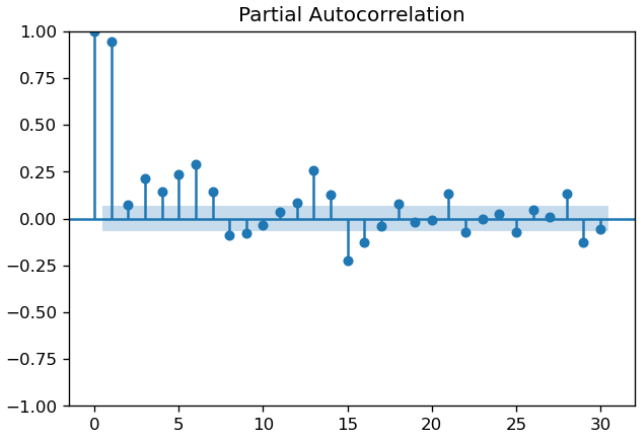
4.2 Model training

a) Non-seasonal ARIMA

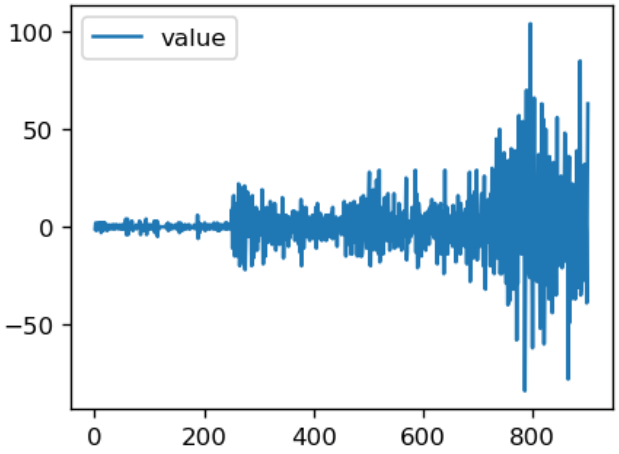
- **Manual estimation of (p, d, q)**

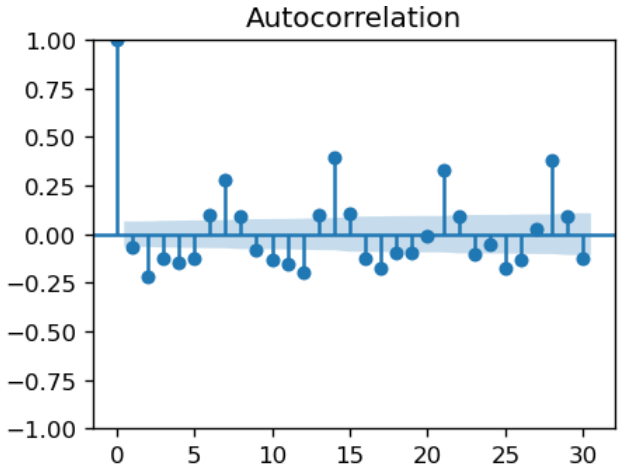
  It is obvious that our time series is nonstationary (it has a strong trend and seasonality). ACF and PACF also show very strong correlation, confirming the non-stationarity of our original data. From the autocorrelation graph, we can decide if more differencing is needed. If collectively the autocorrelations, or the data point of each lag (in the horizontal axis), are positive for several consecutive lags, more differencing might be needed. Conversely, if more data points are negative, the series is over-differenced.

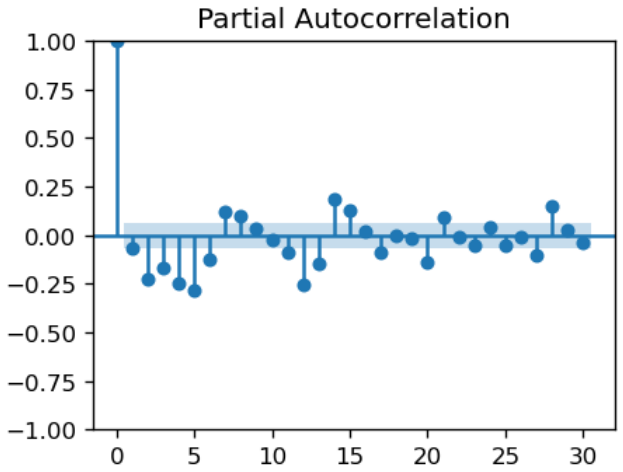




Therefore, differencing is required.
ACF and PACF of 1st differencing show much less correlation:







In the ACF, there is strong correlation for lag 7, 14, 21. This suggests seasonality with a period of 7, which will be fit with the SARIMA model later.

From the graphs, 1st differencing seems to be stationary already.
We can use Augmented Dickey-Fuller test to determine if 1st differencing is enough. A p-value less than 0.05 indicates that the time-series is stationary.

```
# Augmented Dickey-Fuller test (if p < 0.05 --> the series is stationary)
adf_test = adfuller(df_train)
print(f'p-value original series: {adf_test[1]}')

adf_test = adfuller(df_train.diff().dropna())
print(f'p-value 1st differencing: {adf_test[1]}')

adf_test = adfuller(df_train.diff().diff().dropna())
print(f'p-value 2nd differencing: {adf_test[1]}')

# from the above result, 1st differencing is stationaty --> set d = 1
# looking at the ACF & PACF graph of 1st differencing, correlation of all lags are week --> set p = 0 a
```

```
p-value original series: 0.6824788745731261
p-value 1st differencing: 1.4016618701845823e-11
p-value 2nd differencing: 1.4830192174967184e-22
```

The above result confirms that 1st differencing has made our data stationary.

Next, to determine the value of p and q, we look at PACF and ACF graphs, respectively. All lags have weak correlation coefficients (except for lag 7,14,21,... due to seasonality), so we can set p=0 and q=0.

Fitting the ARIMA model to training data with (p, d, q) = (0, 1, 0)

```
# Train non-seasonal ARIMA model:
model = ARIMA(df_train, order=(0,1,0))
model_fit = model.fit()
model_fit.summary()
```

SARIMAX Results

Dep. Variable:	value	**No. Observations:**	902
Model:	ARIMA(0, 1, 0)	**Log Likelihood**	-3702.994
Date:	Sun, 13 Nov 2022	**AIC**	7407.987
Time:	19:40:39	**BIC**	7412.791
Sample:	0	**HQIC**	7409.822
	- 902		
Covariance Type:	opg		

	coef	std err	z	P>|z|	[0.025	0.975]
sigma2	217.4293	4.335	50.152	0.000	208.932	225.927

Ljung-Box (L1) (Q):	4.33	**Jarque-Bera (JB):**	3195.51
Prob(Q):	0.04	**Prob(JB):**	0.00
Heteroskedasticity (H):	22.85	**Skew:**	0.62
Prob(H) (two-sided):	0.00	**Kurtosis:**	12.14

- **Auto-search of (p, d, q)**
  The pmdarima package can search for the optimal combination of (p, d, q).
  Result:

```
# Train auto ARIMA model
auto_arima = pm.auto_arima(df_train,
                           trace=True, stepwise=True, seasonal=False)
```

SARIMAX Results

Dep. Variable:	y	**No. Observations:**	902
Model:	SARIMAX(5, 1, 2)	**Log Likelihood**	-3534.912
Date:	Sun, 13 Nov 2022	**AIC**	7085.824
Time:	23:55:47	**BIC**	7124.252
Sample:	0	**HQIC**	7100.503
	- 902		
Covariance Type:	opg		

	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	0.8053	0.023	34.273	0.000	0.759	0.851
ar.L2	-0.8558	0.028	-30.703	0.000	-0.910	-0.801
ar.L3	-0.0520	0.032	-1.610	0.107	-0.115	0.011
ar.L4	-0.2508	0.027	-9.165	0.000	-0.304	-0.197
ar.L5	-0.1238	0.021	-5.833	0.000	-0.165	-0.082
ma.L1	-1.1443	0.012	-96.813	0.000	-1.167	-1.121
ma.L2	0.9044	0.012	76.641	0.000	0.881	0.927
sigma2	150.1465	3.352	44.790	0.000	143.576	156.717

Ljung-Box (L1) (Q):	0.00	**Jarque-Bera (JB):**	2359.91
Prob(Q):	0.96	**Prob(JB):**	0.00
Heteroskedasticity (H):	18.95	**Skew:**	0.58
Prob(H) (two-sided):	0.00	**Kurtosis:**	10.84

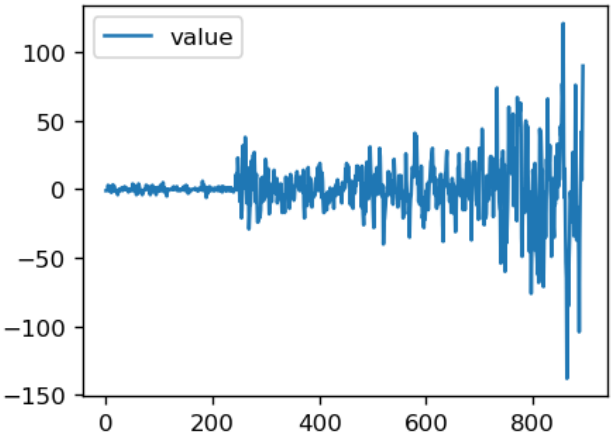
It gives (p, d, q) = (5, 1, 2) which has the same value of d as our manual approach. Having the same d value meaning that we can compare the fitting results.
Log Likelihood, AIC, BIC, HQIC all improved compared to our manual approach.

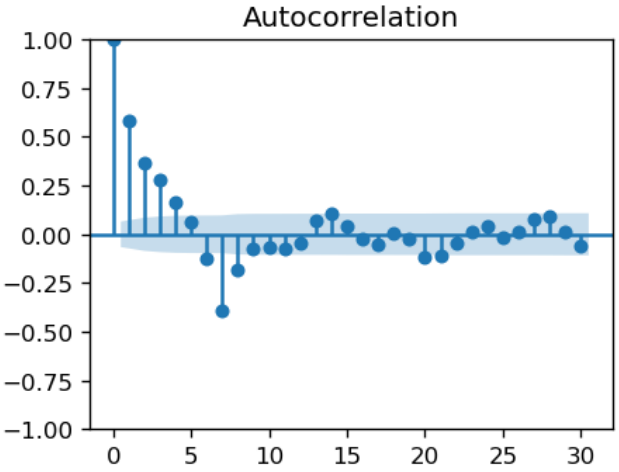
b) Seasonal ARIMA (SARIMA)

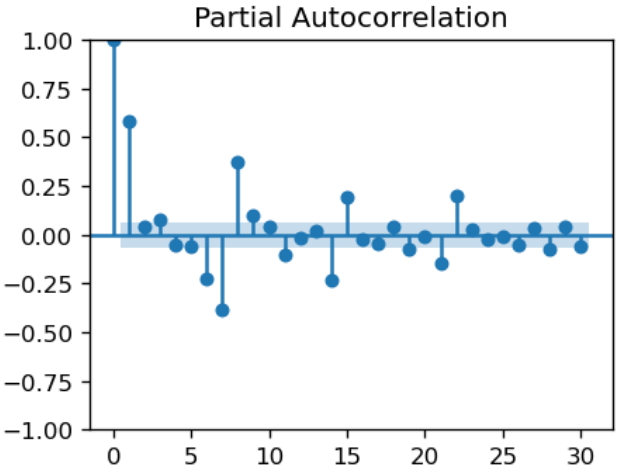
- **Manual estimation of (P, D, Q):**

We will use the same (p, d, q) as our manual approach (0, 1, 0)
To estimate (P, D, Q), we look at the seasonality series (difference between each value and lag 7).

The seasonality series is confirmed to be stationary already → we can set D = 0


value
100
50
0
−50
−100
−150
0
200
400
600
800


Autocorrelation
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
5
10
15
20
25
30


Partial Autocorrelation
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
5
10
15
20
25
30

```
# Augmented Dickey-Fuller test
adf_test = adfuller(df_seasonal)
print(f'p-value original seasonality: {adf_test[1]}')

adf_test = adfuller(df_seasonal.diff().dropna())
print(f'p-value seasonality 1st differencing: {adf_test[1]}')

adf_test = adfuller(df_seasonal.diff().diff().dropna())
print(f'p-value seasonality 2nd differencing: {adf_test[1]}')

# Original seasonality is already stationary --> set D = 0
# Lag 1 in PACF graph is significant --> set P = 1
# Lag 1 to 4 in ACF graph is significant --> set Q = 4
# Cycle happens every 7 days --> set M = 7
```

```
p-value original seasonality: 7.114943259504384e-08
p-value seasonality 1st differencing: 3.2087504162783935e-21
p-value seasonality 2nd differencing: 3.1158536909626216e-23
```

From PACF and ACF, we determine P = 1 and Q = 4
Fitting data:

```
# Train seasonal ARIMA model
model_seasonal = ARIMA(df_train, order=(0,1,0), seasonal_order =(1,0,4,7))
model_fit_seasonal = model_seasonal.fit()
model_fit_seasonal.summary()

# p value > 0.05 for 3rd and 4th MA coefficiency --> reduce q to 2
```

SARIMAX Results

Dep. Variable:	value	**No. Observations:**	902
Model:	ARIMA(0, 1, 0)x(1, 0, [1, 2, 3, 4], 7)	**Log Likelihood**	-3554.240
Date:	Sun, 13 Nov 2022	**AIC**	7120.480
Time:	19:47:16	**BIC**	7149.301
Sample:	0	**HQIC**	7131.489
	- 902		
Covariance Type:	opg		

	coef	std err	z	P>|z|	[0.025	0.975]
ar.S.L7	0.9651	0.009	102.388	0.000	0.947	0.984
ma.S.L7	-0.9416	0.025	-38.037	0.000	-0.990	-0.893
ma.S.L14	0.1772	0.032	5.452	0.000	0.113	0.241
ma.S.L21	-0.0274	0.032	-0.871	0.384	-0.089	0.034
ma.S.L28	0.0117	0.027	0.433	0.665	-0.041	0.065
sigma2	155.1226	3.732	41.570	0.000	147.809	162.436

Ljung-Box (L1) (Q):	34.58	**Jarque-Bera (JB):**	1654.24
Prob(Q):	0.00	**Prob(JB):**	0.00
Heteroskedasticity (H):	17.24	**Skew:**	0.17
Prob(H) (two-sided):	0.00	**Kurtosis:**	9.63

p value > 0.05 for 3rd and 4th MA coefficient --> try reducing Q to 2.
Retraining with Q = 2 does not lead to any significant improvement:

```
# Re-Train seasonal ARIMA model
model_seasonal = ARIMA(df_train, order=(0,1,0), seasonal_order =(1,0,2,7))
model_fit_seasonal = model_seasonal.fit()
model_fit_seasonal.summary()
```

SARIMAX Results

Dep. Variable:	value	**No. Observations:**	902
Model:	ARIMA(0, 1, 0)x(1, 0, [1, 2], 7)	**Log Likelihood**	-3554.435
Date:	Sun, 13 Nov 2022	**AIC**	7116.870
Time:	19:47:33	**BIC**	7136.084
Sample:	0	**HQIC**	7124.210
	- 902		
Covariance Type:	opg		

	coef	std err	z	P>|z|	[0.025	0.975]
ar.S.L7	0.9642	0.009	108.784	0.000	0.947	0.982
ma.S.L7	-0.9393	0.023	-41.206	0.000	-0.984	-0.895
ma.S.L14	0.1623	0.024	6.675	0.000	0.115	0.210
sigma2	155.1932	3.631	42.745	0.000	148.077	162.309

Ljung-Box (L1) (Q):	33.51	**Jarque-Bera (JB):**	1647.37
Prob(Q):	0.00	**Prob(JB):**	0.00
Heteroskedasticity (H):	17.27	**Skew:**	0.17
Prob(H) (two-sided):	0.00	**Kurtosis:**	9.62

- **Auto-search of (P, D, Q)**

```
auto_arima_seasonal = pm.auto_arima(df_train,
                           start_p=0, d=1, start_q=0,
                           max_p=5, max_q=5,
                           start_P=0, D=0, start_Q=0,
                           max_P=5, max_Q=5,
                           error_action = 'warn', m=7,
                           trace=True, stepwise=True, seasonal=True)
```

```
Best model:  ARIMA(5,1,0)(2,0,3)[7]
Total fit time: 278.254 seconds
```

```
auto_arima_seasonal.summary()
```

SARIMAX Results

Dep. Variable:	y	**No. Observations:**	902
Model:	SARIMAX(5, 1, 0)x(2, 0, [1, 2, 3], 7)	**Log Likelihood**	-3492.086
Date:	Mon, 14 Nov 2022	**AIC**	7006.171
Time:	00:06:07	**BIC**	7059.010
Sample:	0	**HQIC**	7026.355
	- 902		
Covariance Type:	opg		

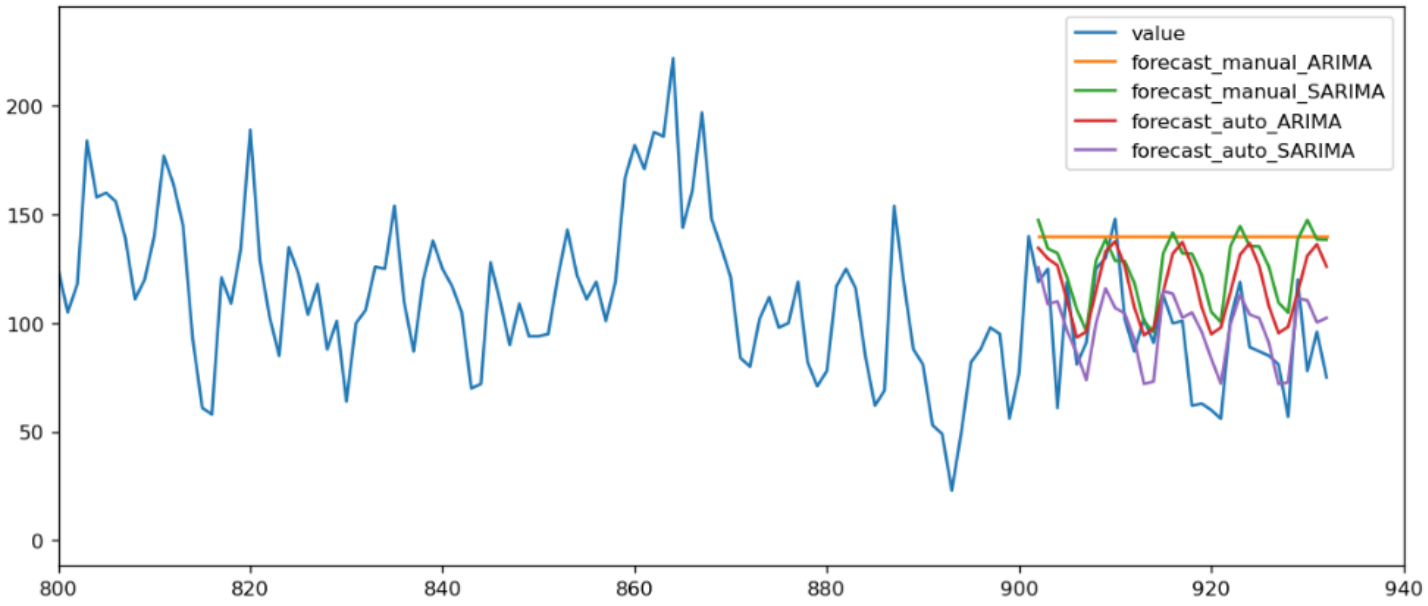
	coef	std err	z	P>|z|	[0.025	0.975]
ar.L1	-0.3005	0.020	-15.145	0.000	-0.339	-0.262
ar.L2	-0.2712	0.026	-10.519	0.000	-0.322	-0.221
ar.L3	-0.1699	0.023	-7.507	0.000	-0.214	-0.126
ar.L4	-0.1711	0.022	-7.936	0.000	-0.213	-0.129
ar.L5	-0.1583	0.022	-7.249	0.000	-0.201	-0.115
ar.S.L7	0.0404	0.042	0.960	0.337	-0.042	0.123
ar.S.L14	0.8984	0.041	21.963	0.000	0.818	0.979
ma.S.L7	-0.0576	0.047	-1.230	0.219	-0.149	0.034
ma.S.L14	-0.6785	0.055	-12.403	0.000	-0.786	-0.571
ma.S.L21	0.0957	0.030	3.146	0.002	0.036	0.155
sigma2	135.0465	3.566	37.869	0.000	128.057	142.036

Ljung-Box (L1) (Q):	0.04	**Jarque-Bera (JB):**	1130.50
Prob(Q):	0.85	**Prob(JB):**	0.00
Heteroskedasticity (H):	17.10	**Skew:**	0.39
Prob(H) (two-sided):	0.00	**Kurtosis:**	8.43

The best combination of (p, d, q) (P, D, Q) is found to be (5, 1, 0) (2, 0, 3)
Log Likelihood, AIC, BIC, HQIC all improved compared to our manual approach.

4.3 Validation

Plotting forecast from 4 different models against the actual data:



Error calculations:

```
mae = mean_absolute_error(df_test, forecast_test_manual_ARIMA)
mape = mean_absolute_percentage_error(df_test, forecast_test_manual_ARIMA)
rmse = np.sqrt(mean_squared_error(df_test, forecast_test_manual_ARIMA))

print(f'mae - manual - ARIMA: {mae}')
print(f'mape - manual - ARIMA: {mape}')
print(f'rmse - manual - ARIMA: {rmse}')
```

```
mae - manual - ARIMA: 46.16129032258065
mape - manual - ARIMA: 0.5924738712629284
rmse - manual - ARIMA: 51.47219731830179
```

```
mae = mean_absolute_error(df_test, forecast_test_auto_ARIMA)
mape = mean_absolute_percentage_error(df_test, forecast_test_auto_ARIMA)
rmse = np.sqrt(mean_squared_error(df_test, forecast_test_auto_ARIMA))

print(f'mae - auto - ARIMA: {mae}')
print(f'mape - auto - ARIMA: {mape}')
print(f'rmse - auto - ARIMA: {rmse}')
```

```
mae - auto - ARIMA: 25.50872487347351
mape - auto - ARIMA: 0.33238012853962373
rmse - auto - ARIMA: 31.75083112002245
```

```
mae = mean_absolute_error(df_test, forecast_test_manual_SARIMA)
mape = mean_absolute_percentage_error(df_test, forecast_test_manual_SARIMA)
rmse = np.sqrt(mean_squared_error(df_test, forecast_test_manual_SARIMA))

print(f'mae - manual - SARIMA: {mae}')
print(f'mape - manual - SARIMA: {mape}')
print(f'rmse - manual - SARIMA: {rmse}')
```

```
mae - manual - SARIMA: 32.71230388197109
mape - manual - SARIMA: 0.4171171195489108
rmse - manual - SARIMA: 38.70189361481308
```

```
mae = mean_absolute_error(df_test, forecast_test_auto_SARIMA)
mape = mean_absolute_percentage_error(df_test, forecast_test_auto_SARIMA)
rmse = np.sqrt(mean_squared_error(df_test, forecast_test_auto_SARIMA))

print(f'mae - auto - SARIMA: {mae}')
print(f'mape - auto - SARIMA: {mape}')
print(f'rmse - auto - SARIMA: {rmse}')
```

```
mae - auto - SARIMA: 17.038138595194035
mape - auto - SARIMA: 0.20780635484351895
rmse - auto - SARIMA: 21.273850733236483
```

In brief, seasonal ARIMA provides better forecasts than non-seasonal ARIMA. This is expected since the original time-series has strong seasonality when decomposed. Similarly, auto search gives better results than manual approach. When the ACF and PACF graphs do not show clear trends, it is better to use auto search for p, q, P, Q.

In terms of errors on validation set, Auto search SARIMA has the smallest error. From best to worst: Auto SARIMA > Auro ARIMA > Manual SARIMA > Manual ARIMA.

Except for Auto SARIMA, the other 3 models over-forecast the quantity because there is a downward trend after December 2021, which is difficult to catch.

5. Discussion and Conclusions

Data Analysis of past performance:

- In general, performance improves over time for all clients, except for **Green is good**. December is the best-performing month in a year.
- **Bol.com Plaza NL** is the best-performing channel overall, whereas **Amazon.de** is in 2nd place for most of the time. The 3rd place position is different each month.
- **Fietsaccessoires** is the best-performing category in terms of orders and quantity, while **Jongenfiets** is in the top spot for revenue. This is reasonable because **Jongenfiets** has higher price than **Fietsaccessoires**.
- Over 90% of orders are shipped, 7% are returned and 1% are cancelled. Percentage of returned orders decreased gradually over the observed period. **Green is good** has very high percentage of returned orders (nearly 30%) compared to other clients (less than 10%). This gives the signal that **Green is good** may need to check the quality of their products, customer service as well as the whole ordering and shipping process.
- Top 3 categories with highest median price are **Elektrische bakfietsen**, **Bakfietsen** and **Cable crossovers**. By checking the time series of Price and Orders/Quantity/Revenue over the short period of time for each category, we can estimate if there is any correlation between Price and Performance.

Forecasting by ARIMA model:

- By combining auto regression, integration technique and moving average, **ARIMA** can predict future values for time series data.
- In general, seasonal models fit the training data better (because our training data has clear seasonality), and auto search outperforms manual approach.
- **Seasonal ARIMA by auto search** gave the best results when validating against the testing set of Product Category “Fietsaccessoires”.
- It is difficult to achieve low error because our data has a strong spike in December 2021. Our last date of training data is 2 January, so it was hard to capture the decreasing trend after December.
- Because there is lots of noise in our daily data, it may be useful to try training and validating new models on weekly data instead of daily data. In contrast, monthly data is probably not a good choice because we have only 3 years of data, so the training set will have fewer than 40 observations of monthly data. Some product categories do not even have more than 12 months of data. With such few observations, it is not possible to capture the trend and seasonality.
- It is also useful to benchmark ARIMA against other forecasting models (eg. Linear regression, xgboost, etc).
- In the future, we can try adding more exogeneous variables into the ARIMA model to capture the holidays.
- For product categories with very low order volume per tenant, a possible solution is to train models on all tenants, then apply the coefficients to each tenant. Another solution is to train models on similar product categories which have higher order volume.